

```
// ----- HC-SR04 Sensors -----
```

```
#define FRONT_TRIG 9
```

```
#define FRONT_ECHO 10
```

```
#define LEFT_TRIG 11
```

```
#define LEFT_ECHO 12
```

```
#define RIGHT_TRIG 13
```

```
#define RIGHT_ECHO A0
```

```
// ----- L293D Motor Driver -----
```

```
#define ENA 5
```

```
#define IN1 7
```

```
#define IN2 8
```

```
#define ENB 6
```

```
#define IN3 4
```

```
#define IN4 3
```

```
// ----- Distance Function -----
```

```
long readDistance(int trigPin, int echoPin)
```

```
{
```

```
    digitalWrite(trigPin, LOW);
```

```
    delayMicroseconds(2);
```

```
    digitalWrite(trigPin, HIGH);
```

```
delayMicroseconds(10);
```

```
digitalWrite(trigPin, LOW);
```

```
long duration = pulseIn(echoPin, HIGH, 30000);
```

```
if (duration == 0)
```

```
    return 300;
```

```
    return duration * 0.034 / 2;
```

```
}
```

```
// ----- Motor Functions -----
```

```
void moveForward()
```

```
{
```

```
    digitalWrite(IN1, HIGH);
```

```
    digitalWrite(IN2, LOW);
```

```
    digitalWrite(IN3, HIGH);
```

```
    digitalWrite(IN4, LOW);
```

```
    analogWrite(ENA, 180);
```

```
    analogWrite(ENB, 180);
```

```
}
```

```
void moveBackward()
{
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);

    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);

    analogWrite(ENA, 180);
    analogWrite(ENB, 180);
}
```

```
void stopMotors()
{
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, LOW);

    digitalWrite(IN3, LOW);
    digitalWrite(IN4, LOW);
}
```

```
void turnLeft()
{
    // Left motor backward
    digitalWrite(IN1, LOW);
    digitalWrite(IN2, HIGH);
```

```
// Right motor forward
digitalWrite(IN3, HIGH);
digitalWrite(IN4, LOW);

analogWrite(ENA, 170);
analogWrite(ENB, 170);

delay(500);

stopMotors();
}
```

```
void turnRight()
{
    // Left motor forward
    digitalWrite(IN1, HIGH);
    digitalWrite(IN2, LOW);

    // Right motor backward
    digitalWrite(IN3, LOW);
    digitalWrite(IN4, HIGH);

    analogWrite(ENA, 170);
    analogWrite(ENB, 170);

    delay(500);
}
```

```
stopMotors();  
}
```

```
// ----- Setup -----
```

```
void setup()
```

```
{
```

```
  Serial.begin(9600);
```

```
  pinMode(FRONT_TRIG, OUTPUT);
```

```
  pinMode(FRONT_ECHO, INPUT);
```

```
  pinMode(LEFT_TRIG, OUTPUT);
```

```
  pinMode(LEFT_ECHO, INPUT);
```

```
  pinMode(RIGHT_TRIG, OUTPUT);
```

```
  pinMode(RIGHT_ECHO, INPUT);
```

```
  pinMode(ENA, OUTPUT);
```

```
  pinMode(IN1, OUTPUT);
```

```
  pinMode(IN2, OUTPUT);
```

```
  pinMode(ENB, OUTPUT);
```

```
  pinMode(IN3, OUTPUT);
```

```
  pinMode(IN4, OUTPUT);
```

```
    stopMotors();
}

// ----- Main Logic -----

void loop()
{
    // Normally only check FRONT
    long frontDistance = readDistance(FRONT_TRIG, FRONT_ECHO);

    Serial.print("FRONT: ");
    Serial.print(frontDistance);
    Serial.println(" cm");

    // ----- PATH CLEAR -----

    if (frontDistance >= 20)
    {
        moveForward();
    }

    // ----- OBSTACLE AHEAD -----

    else
    {
```

```
stopMotors();
```

```
Serial.println("OBSTACLE AHEAD!");
```

```
delay(300);
```

```
// Now check LEFT and RIGHT
```

```
long leftDistance = readDistance(LEFT_TRIG, LEFT_ECHO);
```

```
delay(50);
```

```
long rightDistance = readDistance(RIGHT_TRIG, RIGHT_ECHO);
```

```
Serial.print("LEFT: ");
```

```
Serial.print(leftDistance);
```

```
Serial.print(" cm | RIGHT: ");
```

```
Serial.print(rightDistance);
```

```
Serial.println(" cm");
```

```
// ----- LEFT CLEARER -----
```

```
if (leftDistance > rightDistance && leftDistance >= 20)
```

```
{
```

```
    Serial.println("LEFT PATH CLEAR -> TURNING LEFT");
```

```
    turnLeft();
```

```
// After turning, start moving  
moveForward();  
}
```

```
// ----- RIGHT CLEARER -----
```

```
else if (rightDistance >= 20)
```

```
{
```

```
  Serial.println("RIGHT PATH CLEAR -> TURNING RIGHT");
```

```
  turnRight();
```

```
  // After turning, start moving
```

```
  moveForward();
```

```
}
```

```
// ----- BOTH SIDES BLOCKED -----
```

```
else
```

```
{
```

```
  Serial.println("LEFT & RIGHT BLOCKED -> REVERSING");
```

```
  moveBackward();
```

```
  delay(700);
```



```
    stopMotors();  
  }  
}  
  
  delay(100);  
}
```